GestureClick

Stage 1: The Live Webcam Pipeline

Camera → landmarks → normalized hand, running in real time

Difficulty	Moderate (2 of 4)
Time	about 3–4 sessions of 30–60 minutes
You will write	main.py, plus small fixes in Gesture_landmark.py and normalizer.py
Done when	you wave at the camera, see the landmarks follow your hand at 15+ FPS, your hand can leave the frame without a crash, and q closes everything cleanly

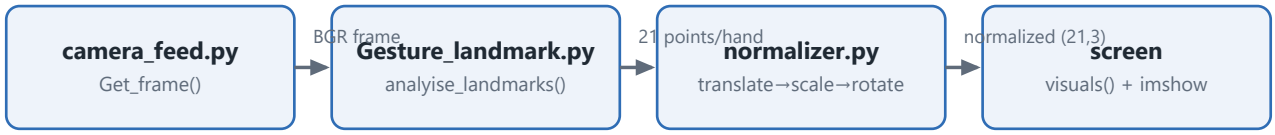
Contents

1. Why this stage exists
2. The concepts you need first
3. Before you start: two fixes
4. The steps (1 to 8)
5. Final test checklist
6. Troubleshooting
7. Glossary
8. What comes next

> **How to use this guide.** Do the steps in order. Every step ends with a **Check**. Do not start the next step until that check passes. If something breaks, you then know it was the last thing you added. The code pieces are building blocks: you put them together in your own main.py.

1. Why this stage exists

Until now your code analysed **one saved picture**, printed some numbers and stopped. GestureClick has to watch the camera **continuously** and react to your hand while it moves. This stage builds that running program. Every later stage (features, gesture recognition, taking the photo) runs *inside* the loop you build here, once per frame.



main.py runs all of this once per frame, about 15–30 times every second

Now	After this stage
reads one saved .jpg	reads the webcam continuously
runs once, prints, exits	runs until you press q
timestamp=1 (a fake time)	real time in milliseconds
you read printed numbers	you see your hand and its landmarks live

Why do it before the features stage?

- **Real test data.** A few saved photos cannot show every position, distance, tilt and lighting. The live camera shows you hundreds in seconds.
- **Real problems show up.** Frames with no hand, a hand half outside the picture, blur, flicker, low speed. Your code has to survive all of these.
- **Live proof of your normalization.** You will watch the middle knuckle stay near [1, 0, z] while you move and tilt your hand.

- **It sets up time.** Stage 4 (hold a gesture for 0.5 s, then wait before the next photo) is built on the clock you add here.

2. The concepts you need first

2.1 A video is just many pictures

A webcam does not send "video". It sends separate pictures called **frames**, very quickly, usually about 30 per second. Each frame is a NumPy array of shape `(height, width, 3)`, for example `(480, 640, 3)`: 480 rows of pixels, 640 columns, and 3 colour values per pixel.

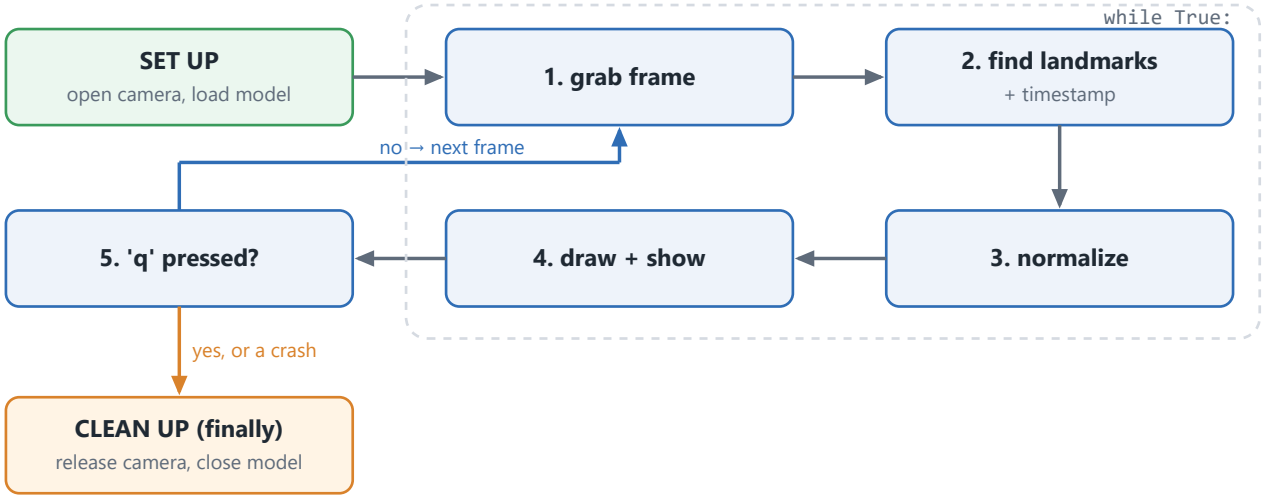
FPS (frames per second) measures how many frames your program handles each second. If your processing is slow, FPS drops and gestures feel laggy. Below about 10 FPS it feels broken.

2.2 BGR, not RGB

OpenCV stores colour in the order **Blue, Green, Red**. MediaPipe expects Red, Green, Blue. Your `analyise_landmarks()` already converts with `cv.cvtColor(frame, cv.COLOR_BGR2RGB)`, so you only need to remember this when you choose colours for drawing: `(0, 255, 0)` is green, and `(0, 0, 255)` is **red**, not blue.

2.3 The loop

Every real-time video program has the same shape:



Two OpenCV calls do the showing and the key check:

- `cv.imshow("GestureClick", frame)` puts the frame in a window.
- `cv.waitKey(1)` waits 1 millisecond for a key press. It is also the call that actually **repaints the window**. Without it the window stays grey.
- `cv.waitKey(1) & 0xFF == ord('q')` means "was the key q?". `ord('q')` is the number for the letter q, and `& 0xFF` keeps only the last 8 bits of the key code, which makes it work the same on every system.

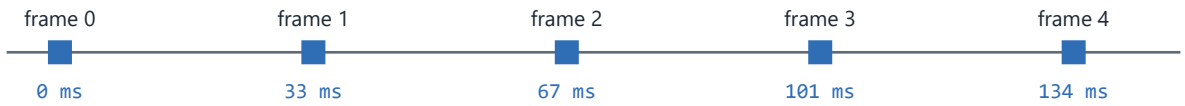
2.4 try / finally: always clean up

The camera is a shared device. If your program crashes while holding it, Windows may think it is still in use (the camera light stays on), and the next run cannot open it. Code in a `finally:` block runs **whatever happens**: normal exit, q pressed, or a crash.

```
try:
    while True:
        ...            # the loop
finally:
    camera.terminate()     # always runs, even after an error
    tracker.terminate()
```

2.5 Timestamps and VIDEO mode

Your Handtriangulator uses MediaPipe's **VIDEO mode**. In this mode MediaPipe does not search the whole picture every time. It **tracks** the hand from the previous frame, which is faster and smoother. To track, it must know *when* each frame happened, so every call needs a timestamp in milliseconds.



each timestamp is larger than the one before (must always increase)

> **The rule:** every timestamp must be **larger than the previous one**. Passing `timestamp=1` on every frame, as in your test code, makes MediaPipe raise an error on the second frame, because time cannot stand still.

Measure real time like this:

```
import time
start = time.monotonic()                          # once, before the loop
timestamp_ms = int((time.monotonic() - start) * 1000)   # inside the loop, every frame
```

- `time.monotonic()` is a clock that **only goes forwards**. `time.time()` is the wall clock, and it can jump backwards (clock sync, summer time), which would break MediaPipe.
- Subtracting `start` gives time since the program began, in seconds. `* 1000` converts to milliseconds, and `int()` makes it a whole number, which MediaPipe requires.

2.6 How main.py finds your other files

Python looks for imports in a list of folders called `sys.path`. When you run `main.py`, its own folder (`gesture_click/`) is on that list, but `gesture_click/source/` is not. So `from camera_feed import Camera_Feed` fails with `ModuleNotFoundError`. The fix is to add the source folder to the list, built from main.py's own location (the same `__file__` trick you used for DATA_DIR):

```
import sys
from pathlib import Path
SOURCE_DIR = Path(__file__).resolve().parent / "source"
sys.path.insert(0, str(SOURCE_DIR))     # must come BEFORE importing your own modules
```

This is also why the file had to be renamed from camera-feed.py to camera_feed.py: a module name cannot contain -, because Python reads it as a minus sign.

2.7 Frames without a hand

When no hand is visible, `result.hand_landmarks` is an **empty list** `[]`. That is normal, not an error. Your normalizer already returns `[]` for it. Every part of the loop must accept "zero hands" without crashing.

3. Before you start: two fixes

Fix A: the typo in normalizer.py

In `translation_invariance` the finite check says `np.isinfinite`. That function does not exist, so the file crashes. The correct name is `np.isfinite` ("is finite").

Check: `python gesture_click\source\normalizer.py` prints arrays again.

Fix B: visuals() returns nothing when there is no hand

In `Gesture_landmark.py`, `return frame` sits **inside** `if generation.hand_landmarks:`. With no hand, the method reaches its end without a return, and Python then returns `None`. Your loop would pass `None` to `cv.imshow`, and the program would crash the moment your hand leaves the picture.

```
# now (wrong)                              # fixed
if generation.hand_landmarks:              if generation.hand_landmarks:
    for hand_landmarks in ...:                 for hand_landmarks in ...:
        drawing_utils.draw_landmarks(...)          drawing_utils.draw_landmarks(...)
    self.annotted = frame                  self.annotted = frame
    return frame                           return frame          # <- one level LEFT
```

- Also change the return hint from `-> None` to `-> np.ndarray`, because the method returns a picture.
- **General rule:** every path through a function should end in a deliberate `return`. "Falling off the end" silently returns `None`.

4. The steps

All code in this section goes into `gesture_click/main.py` unless the step says otherwise.

STEP 1 Imports and paths

about 5 min

Goal: main.py can import your three modules.

```
import sys
import time
from pathlib import Path

import cv2 as cv
import numpy as np

SOURCE_DIR = Path(__file__).resolve().parent / "source"
sys.path.insert(0, str(SOURCE_DIR))

from camera_feed import Camera_Feed
from Gesture_landmark import Handtriangulator, MODEL_PATH
from normalizer import Normalizer, MIDDLE_MCP
```

Why the order matters: `sys.path.insert` must run before the `from camera_feed ...` lines, otherwise Python has not been told where to look yet.

Check: run main.py. Nothing should happen, and there should be **no ImportError**.

STEP 2 The bare camera loop (no MediaPipe yet)

about 10 min

Goal: see yourself in a window and close it with q. Build this **before** adding MediaPipe, so that if something fails you know it is the camera part.

```
camera = Camera_Feed()

try:
    while True:
        frame = camera.Get_frame()
        cv.imshow("GestureClick", frame)
        if cv.waitKey(1) & 0xFF == ord('q'):
            break
finally:
    camera.terminate()      # releases the camera and closes windows
```

- `while True:` repeats forever; `break` is the only way out.
- `Camera_Feed.terminate()` already calls `cap.release()` and `cv.destroyAllWindows()`, so the clean-up is one line.

Check: a window shows the live camera, and pressing **q** (with the window selected) closes it and turns the camera light off.

Windows tip: if the camera takes several seconds to open, try `cv.VideoCapture(camera_index, cv.CAP_DSHOW)` inside Camera_Feed. DirectShow often starts faster than Windows' default camera system.

STEP 3 Landmarks with real timestamps

about 10 min

Goal: the hand skeleton is drawn on the live video.

Before the loop:

```
tracker = Handtriangulator(model_path=str(MODEL_PATH))
start = time.monotonic()
```

Inside the loop, between Get_frame and imshow:

```
timestamp_ms = int((time.monotonic() - start) * 1000)
result = tracker.analyise_landmarks(frame, timestamp_ms)
frame = tracker.visuals(frame, result)
```

And in `finally:` add `tracker.terminate()`.

- `analyise_landmarks` already accepts a NumPy frame and converts BGR to RGB.
- `visuals` draws on the frame and returns it (after Fix B, also when there is no hand).
- The model is created **once**, before the loop. Creating it inside the loop would reload the model file 30 times a second.

Check: (1) the skeleton follows your hand; (2) move your hand out of the picture and back: no crash; (3) both hands get skeletons.

STEP 4 Run the normalizer live

about 10 min

Goal: prove that translation, scale and rotation work on live data.

Before the loop: `normalizer = Normalizer()` and `frame_count = 0`. Inside the loop, after `analyise_landmarks`:

```
frame_count += 1
try:
    translated = normalizer.translation_invariance(result)
    scaled = normalizer.scale_invariance(translated)
    rotated = normalizer.rotational_invariance(scaled)
except RuntimeError as e:
    print("skipped frame:", e)
    rotated = []

if frame_count % 15 == 0:
    for hand in rotated:
        print(np.round(hand[MIDDLE_MCP], 3))
```

Why the try/except here?

In a live program a single strange frame (for example a detection that fails your zero-scale guard) should not end the whole program. Catching `RuntimeError` skips just that frame. It still **prints** the reason, so problems are not hidden. Only catch the error type you expect: a bare `except:` would also hide real bugs.

Why print only every 15th frame?

`%` gives the remainder of a division, so `frame_count % 15 == 0` is true on frames 15, 30, 45 and so on: about twice a second at 30 FPS. Printing every frame would flood the terminal and also slow the loop down.

> **Check (the payoff of the whole last chapter):** the printed value stays close to `[1.0, 0.0, small]` while you move your hand around the picture, closer and further, and tilted. Position, size and tilt no longer change the numbers.

STEP 5 Show the FPS

about 10 min

Goal: know how fast your loop runs, because every later stage adds work to it.

One frame's FPS is `1 / (time spent on that frame)`. It jumps around a lot, so average the last 10 frame times. A deque with `maxlen` is a list that automatically drops its oldest item when full, which is ideal for "the last N values".

```
from collections import deque
frame_times = deque(maxlen=10)
previous = time.monotonic()                    # before the loop

# inside the loop:
now = time.monotonic()
frame_times.append(now - previous)
previous = now
fps = len(frame_times) / sum(frame_times)
cv.putText(frame, f"FPS: {fps:.0f}", (10, 30),
           cv.FONT_HERSHEY_SIMPLEX, 1, (0, 255, 0), 2)
```

`putText` arguments: the frame, the text, the position of the text's bottom-left corner in pixels, the font, the size, the colour in **BGR** (so this is green), and the line thickness. Draw it **after** `visuals()` so it

appears on top.

Check: 15 or more FPS with one hand visible. If it is much lower, check that the model is created only once (Step 3) and that you are not printing every frame.

STEP 6 The aspect-ratio fix (in normalizer.py)

about 15 min

Goal: make x and y use the same unit, so distances and angles are correct.

The problem

MediaPipe gives `x = pixel_x / width` and `y = pixel_y / height`. On a 640 × 480 webcam, one unit of x is 640 pixels long but one unit of y is only 480 pixels. The two axes use different rulers, so a perfect circle in the picture would come out as an oval in your numbers, and your rotation step would slightly stretch the hand.

The maths

Multiply x by `width / height`:

```
x_corrected = x * (width / height)
            = (pixel_x / width) * (width / height)
            = pixel_x / height          # now x is measured with the same ruler as y
```

The two `width` values cancel. Both axes are now "fractions of the picture height".

Where to put it

In `translation_invariance`, because it is the first step and everything after it must use consistent units. Add a parameter `aspect: float = 1.0` and multiply the x line:

```
x = (landmark.x - reference.x) * aspect
```

- Multiplying **after** subtracting gives the same result as multiplying both points first, because `a·k - b·k = (a - b)·k`. So you only need to change one line.
- The default `1.0` means "no change", so your existing test code keeps working.
- In main.py compute it from the frame: `height, width = frame.shape[:2]` (note: **height comes first** in NumPy image shapes), then pass `aspect=width / height`.

Check: tilt your hand slowly from upright to sideways. The printed x of the middle knuckle should stay close to 1.0 at every angle. Without the fix, it drifts by several percent as you tilt.

STEP 7 Decide: mirror the picture or not

about 5 min

A webcam shows you as other people see you, so raising your right hand looks like the left on screen. Most camera apps flip the picture so it behaves like a mirror:

```
frame = cv.flip(frame, 1)      # 1 = flip left-right; do it right after Get_frame
```

Important: flipping changes which hand MediaPipe calls "Left" and "Right". MediaPipe's documentation says its handedness assumes a mirrored (selfie) picture, so after flipping, the labels should match your real hands. **Verify it:** raise only your right hand and print `result.handedness[0][0].category_name`. Then **write your decision in Notes and never change it**: custom gestures saved later depend on it.

STEP 8 **Tidy up and review** about 5 min

- Remove any temporary prints you no longer need (keep the every-15th-frame check for now).
- Make sure `finally:` closes both the camera and the tracker.
- Run once more, then work through the checklist in section 5.

5. Final test checklist

Stage 1 is finished when all of these are true:

#	Test	Expected
1	Start main.py	window with live video opens within a few seconds
2	Show one hand	skeleton drawn and follows the hand smoothly
3	Show two hands	two skeletons
4	Move hand out of the picture and back	no crash; drawing stops and restarts
5	Look at the FPS text	15 or more
6	Move hand around, near and far, tilted	middle knuckle prints stay near [1.0, 0.0, small]
7	Tilt slowly (aspect fix)	x stays close to 1.0 at every angle
8	Raise right hand only	label matches the decision written in Notes
9	Press q	window closes, camera light goes off
10	Run main.py again straight away	camera opens normally (clean-up worked)

6. Troubleshooting

Symptom	Likely cause	Fix
ModuleNotFoundError: camera_feed	source folder not on sys.path, or insert comes after the import	Step 1: put sys.path.insert above your imports
MediaPipe error mentioning the timestamp	timestamp not increasing, or not an int	Section 2.5: time.monotonic(), * 1000, int()
Window grey or frozen	cv.waitKey(1) missing	Step 2
Crash when the hand leaves the picture	visuals() returns None	Fix B in section 3
Camera "in use" after a crash	clean-up not in finally	Section 2.4
Very slow (under 10 FPS)	model created inside the loop, or printing every frame	Step 3 and Step 4
"skipped frame: ... about zero"	a strange detection hit your scale guard	normal now and then; worry only if it happens constantly
Colours look wrong in your text	BGR order	(0, 0, 255) is red in OpenCV
Left/Right labels swapped	picture flipped or not flipped	Step 7: decide once and write it down
q does nothing	the terminal has keyboard focus, not the video window	click the video window first

7. Glossary

Term	Meaning
frame	one picture from the camera; a NumPy array (height, width, 3)
FPS	frames per second: how many frames your loop handles each second
BGR	OpenCV's colour order: Blue, Green, Red
VIDEO mode	MediaPipe mode that tracks the hand between frames; needs increasing timestamps
timestamp	when a frame happened, in milliseconds since the program started
time.monotonic()	a clock that never goes backwards
try / finally	finally runs no matter what, so it is used for clean-up
sys.path	the list of folders Python searches when you import something
deque(maxlen=N)	a list that keeps only the newest N items
%	remainder after division; frame_count % 15 == 0 means every 15th frame
aspect ratio	width / height of the picture; 640/480 = 1.333
handedness	MediaPipe's guess of Left or Right hand, depends on mirroring

8. What comes next

After Stage 1 your program runs end to end: **camera** → **landmarks** → **normalized hand**. The next stage, **Features**, turns each normalized hand into a few meaningful measurements: is each finger straight or bent, how far is the thumb tip from the index tip, which way does the thumb point. The new maths there is the **dot product**, used to measure the angle at each finger joint. You will test every feature live, in the loop you just built.

> Also still open from the normalization chapter: **test_normalizer.py** and writing the four limitations in Notes. The aspect-ratio fix in Step 6 removes one of those limitations: remember to update the note.